```
!pip install -q gradio transformers torch black autopep8 reportlab requests
```

```
import re
import sqlite3
import datetime
import urllib.parse

import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

import gradio as gr
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import letter

print("Libraries loaded successfully")
```

```
model_name = "ibm-granite/granite-3.3-2b-instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    torch_dtype="auto"
)

print("Model Loaded Successfully")
```

```
prompt = "Convert this request to SQL: show all users"

inputs = tokenizer(prompt, return_tensors="pt").to("cuda")

output = model.generate(**inputs, max_new_tokens=60)

print(tokenizer.decode(output[0]))
```

```
DANGEROUS_KEYWORDS = [
    "DROP",
    "DELETE",
    "ALTER",
    "TRUNCATE",
    "INSERT",
    "UPDATE",
    "EXEC",
    "EXECUTE"
]
```

```
DANGEROUS_PATTERNS = [
    ";",      # command chaining
    "--",     # SQL comment
    "/*",     # hidden comments
    "*/",
    "xp_",    # system procedures
    "sp_"
]
```

```
def is_safe(user_input):

    text = user_input.upper()

    # check dangerous keywords
    for keyword in DANGEROUS_KEYWORDS:
        if keyword in text:
            return False

    # check dangerous patterns
    for pattern in DANGEROUS_PATTERNS:
        if pattern in text:
            return False

    return True
```

```
print(is_safe("show all users"))
print(is_safe("DROP TABLE users"))
```

```
schema = {

    "users": {
        "id": "INTEGER",
        "name": "TEXT",
        "email": "TEXT",
        "signup_date": "DATE",
        "age": "INTEGER",
        "country": "TEXT",
        "status": "TEXT"
    },

    "orders": {
        "id": "INTEGER",
        "user_id": "INTEGER",
        "product_name": "TEXT",
        "amount": "REAL",
        "order_date": "DATE",
        "status": "TEXT"
    },

    "products": {
        "id": "INTEGER",
        "name": "TEXT",
        "price": "REAL",
        "category": "TEXT",
        "stock": "INTEGER"
    },

    "transactions": {
        "id": "INTEGER",
        "user_id": "INTEGER",
        "amount": "REAL",
        "date": "DATE",
        "type": "TEXT"
    }
}
```

```
print(schema.keys())
```

```
def get_table_name(query):

    query = query.lower()

    if any(word in query for word in ["user","users","customer","customers","account","accounts","profile"]):
        return "users"

    elif any(word in query for word in ["order","orders","purchase","purchases","buy","bought"]):
        return "orders"

    elif any(word in query for word in ["product","products","item","items","goods"]):
        return "products"

    elif any(word in query for word in ["transaction","transactions","payment","payments","transfer"]):
        return "transactions"
```

```
    else:
        return None
```

```
print(get_table_name("show all users"))
print(get_table_name("find transactions greater than 500"))
print(get_table_name("list products"))
```

```
def get_column_names(query, table):

    query = query.lower()

    columns = []

    column_mapping = {

        "name": ["name","username","full name"],
        "email": ["email","mail","address"],
        "signup_date": ["signup","joined","registered","date","month"],
        "age": ["age","years"],
        "amount": ["amount","price","cost","total"],
        "status": ["status","state"],
        "country": ["country","location","place"]
    }

    for column, keywords in column_mapping.items():

        if any(word in query for word in keywords):
            columns.append(column)

    if not columns:
        columns.append("*")

    return columns
```

```
print(get_column_names("show name and email of users", "users"))

print(get_column_names("find transactions with amount", "transactions"))

print(get_column_names("list all products", "products"))
```

```
import re

def extract_conditions(query):

    query = query.lower()
    conditions = []

    # detect "greater than number"
    match = re.search(r'greater than (\d+)', query)
    if match:
        value = match.group(1)
        conditions.append(f"amount > {value}")

    # detect "last week"
    if "last week" in query:
        conditions.append("DATE(date) >= DATE('now','-7 days')")

    # detect "last month"
    if "last month" in query:
        conditions.append("DATE(signup_date) >= DATE('now','-1 month')")

    return conditions
```

```
print(extract_conditions("find transactions greater than 500"))

print(extract_conditions("show users who signed up last month"))

print(extract_conditions("transactions greater than 1000 last week"))
```

```
def generate_sql(query):

    # Step 1: Security validation
```

```
    if not is_safe(query):
        return "❌ Unsafe query detected"

    # Step 2: Detect table
    table = get_table_name(query)

    if table is None:
        return "⚠ Could not detect table"

    # Step 3: Detect columns
    columns = get_column_names(query, table)

    # Step 4: Extract conditions
    conditions = extract_conditions(query)

    # Step 5: Build SQL query
    sql = f"SELECT {', '.join(columns)} FROM {table}"

    if conditions:
        sql += " WHERE " + " AND ".join(conditions)

    sql += " LIMIT 10"

    return sql
```

```
print(generate_sql("show users who signed up last month"))

print(generate_sql("find transactions greater than 500"))

print(generate_sql("show name and email of users"))
```

```
import gradio as gr

def chat_interface(user_input):
    return generate_sql(user_input)

demo = gr.Interface(
    fn=chat_interface,
    inputs="text",
    outputs="text",
    title="AI SQL Query Generator",
    description="Convert natural language to SQL queries"
)

demo.launch(share=True)
```

```
query_history = []
```

```
def chat_interface(user_input):

    sql_query = generate_sql(user_input)

    query_history.append({
        "request": user_input,
        "sql": sql_query
    })

    return sql_query
```

```
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import letter
import datetime

def export_pdf():

    filename = "sql_report_" + datetime.datetime.now().strftime("%Y%m%d_%H%M%S") + ".pdf"

    c = canvas.Canvas(filename, pagesize=letter)

    y = 750

    c.setFont("Helvetica", 12)

    c.drawString(50, 800, "SQL Query Generator Report")
```

```
    for item in query_history:

        c.drawString(50, y, "User Request: " + item["request"])
        y -= 20

        c.drawString(50, y, "Generated SQL: " + item["sql"])
        y -= 40

    c.save()

    return filename
```

```
export_pdf()
```

```
from google.colab import files
files.download(export_pdf())
```

```
print(query_history)
```

```
chat_interface("show users who signed up last month")
chat_interface("find transactions greater than 500")
chat_interface("show name and email of users")
```

```
print(query_history)
```

```
export_pdf()
```

```
import urllib.parse

def whatsapp_share(sql_query):

    message = urllib.parse.quote(sql_query)

    link = f"https://wa.me/?text={message}"

    return link
```

```
query = generate_sql("show users who signed up last month")

print(whatsapp_share(query))
```

```
demo.launch(share=True)
```